Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Medyan bulma tekrarı
  - Böl ve fethet ile medyan bulmada pivot medyanların medyanı yöntemiyle seçilir. Dizi 5'erli sütunlara ayrılır; her sütunun sıralanması $O(1)$'dir, $n/5$ sütun olduğu için bu iş toplamda $O(n)$ tutar.
  - Algoritma pivotun yalnız bir tarafıyla devam eder, öbür parçayı atar. Recurrence yazılırken kalan parçanın en büyük olabileceği durum alınır: $7n/10 + 6$. Medyanların medyanı için de $T(\lceil n/5 \rceil)$ gelir.
  - Bu bağıntıda alt problemler eşit boyutlu olmadığı için master teoremi uygulanamaz; özyineleme ağacı da çok zorlaşır. Bu yüzden tümevarımlı yerine koyma yöntemi kullanılır.
- Tümevarımla yerine koyma yöntemi (substitution method that uses mathematical induction)
  - 1. adım: çözümün biçimi tahmin edilir (örneğin $T(n) \leq cn \log n$). Bu adım yöntemin zor tarafıdır.
  - 2. adım: tahminin n'den küçük bütün m değerleri için doğru olduğu kabul edilir ($T(m) \leq cm \log m$, $m < n$) ve n için doğruluğu gösterilir. Ardından uygun c ve n_0 bulunur.
  - Master teoremi kadar pratik değildir, hataya açıktır; master teoremi ya da özyineleme ağacıyla çözülemeyen durumlarda işe yarar.
- Merge sort örneği (ekrandan, 00:26:40)
  - $T(n) = 2T(\lfloor n/2 \rfloor) + n$, $T(1) = 1$. Tahmin $T(n) = O(n \log n)$, yani $T(n) \leq cn \log n$.
  - $m = n/2$ için varsayım yerine konur: $T(n) \leq 2c\frac{n}{2} \log \frac{n}{2} + n = cn(\log n - 1) + n = cn \log n - n(c-1) \leq cn \log n$.
  - $n_0 = 1$ olmaz: $T(1) = 1$ ama $c \cdot 1 \cdot \log 1 = 0$. $n \geq 2$ alınır: $T(2) = 4$, $T(3) = 5$. $c = 1$ ile $4 \leq 2$ sağlanmaz; $c = 2$ ile $4 \leq 4$ sağlanır. Sonuç: $c = 2$, $n_0 = 2$ için $T(n) \leq cn \log n$.
- Yanlış tahmin örneği (ekrandan, 00:31:30)
  - Aynı bağıntı için $T(n) = O(n)$, yani $T(n) \leq cn$ tahmin edilirse $T(n) \leq 2c\frac{n}{2} + n = cn + n$ çıkar. n pozitif olduğundan $cn + n \leq cn$ hiçbir zaman sağlanmaz; tahmin yanlıştır.
- Medyan bulma örneği (ekrandan, 00:40:00 ve 00:42:40)
  - $T(n) = O(1)$ ($n < 120$), $T(n) = T(\lceil n/5 \rceil) + T(7n/10+6) + O(n)$ ($n \geq 120$). Tahmin $T(n) \leq cn$; varsayım $T(n/5) \leq cn/5$ ve $T(7n/10 + 6) \leq c(7n/10 + 6)$. Doğrusal terim an diye yazılır.
  - $T(n) \leq cn/5 + 7cn/10 + 6c + an = 9cn/10 + 6c + an = cn - (cn/10 - 6c - an)$.
  - Gereken koşul $cn/10 - 6c - an \geq 0$, yani $c \geq 60c/n + 10a$. $n \geq 120$ ve $c \geq 20a$ seçilirse $20a \geq \frac{60 \cdot 20a}{120} + 10a = 20a$ sağlanır. Böylece $T(n) = O(n)$ gösterilir.
- Bağlı liste tekrarı
  - Düğümler (node) birbirini işaret eder; ekleme ve silme kolaydır, boyut dinamik değişir. Son düğüm NULL'ı gösterir.
  - Tek yönlü bağlı listede (singly linked list) yalnız ileri gidilir. Ekleme: yeni düğüm oluşturulur, önceki düğümün next'i yeni düğüme, yeni düğümün next'i eski sonrakine bağlanır. Silme: A'nın next'i X yerine B'yi gösterir.
  - C kodları gösterildi: sona ekleme, başa ekleme (yeni head döndürülür), listeleme

ve değere göre silme (ekrandan, 01:00:00). Silmede değer verildiği için düğüm aranır; önceki düğüm (`before`) de tutulur, `before->next = current->next` yapılır ve `free(current)` ile bellek bırakılır.
  - Çift yönlü bağlı listede (doubly linked list) her düğüm hem öncekini hem sonrakini gösterir; silmede iki bağlantı birden güncellenir.
- Sözlük (dictionary) ve doğrudan adresleme (direct addressing)
  - Sözlük: search, insert, delete işlemlerinin tanımlı olduğu dinamik küme. Kayıtlarda bir alan anahtar (key) olarak varlığı ayırt eder.
  - Hashing, anahtarları bir boyutlu bir diziye dağıtma fikridir. Hash fonksiyonu anahtara 0 ile $m-1$ arasında bir tam sayı (hash adresi) atar. Kullanım: derleyicilerde sembol tabloları, kriptoloji.
  - Doğrudan adresleme (ekrandan, 01:10:00): evren U küçükse ve iki eleman aynı anahtarı taşımıyorsa $T[k]$ doğrudan anahtarla adreslenir; search, insert, delete $O(1)$. Sakıncası: U büyükse $|U|$ boyutlu tablo pratik değildir; saklanan anahtar kümesi K küçükse alanın çoğu boşa gider.
  - Doğrudan tabloda en büyük elemanı bulmak gibi bir işlemde bütün tablo taranır; en kötü durum $O(m)$.
- Hashing ve çakışma (collision)
  - Eleman, anahtarın kendisinin değil $h(k)$'nin gösterdiği slota konur; tablo evrenden küçük olabilir. İki farklı anahtar aynı slota düşerse çakışma olur.
  - İyi bir hash fonksiyonu rastgele görünümlü (random looking) dağıtır; en basit hâllerde asal sayılardan yararlanılır. Çakışmalar yine de olur ve çözülmesi gerekir.
- Zincirleme (chaining)
  - Aynı slota düşen elemanlar bir bağlı listede tutulur; slot listenin başını gösterir. Çift yönlü bağlı liste kullanılır, çünkü silmeyi hızlandırır.
  - Ekleme yeni elemanı listenin başına koyarak $O(1)$ yapılır. Silme, düğümün adresi verildiğinde çift yönlü listede $O(1)$'dir; tek yönlü listede önceki düğüm bilinmediği için $O(n)$ olur.
  - Bütün anahtarlar aynı slota düşerse (kötü hash fonksiyonu ya da kasıtlı girdi) arama $O(n)$ olur; en iyi durum $O(1)$.
- Zincirlemeli hashing analizi (ekrandan, 01:20:00 ve 01:40:00)
  - T tablo, m slot sayısı, n saklanan eleman sayısı. Yük faktörü (load factor) $\alpha = n/m$: bir zincirdeki ortalama eleman sayısı. α 1'den küçük, eşit ya da büyük olabilir.
  - Basit düzgün hashing (simple uniform hashing): her elemanın m slottan herhangi birine düşme olasılığı eşittir.
  - Teorem: başarısız arama ortalama $\Theta(1+\alpha)$ sürer; arama zincirin sonuna kadar gider, zincirde ortalama α eleman vardır.
  - Teorem (ekrandan, 01:33:40): başarılı arama da ortalama $\Theta(1+\alpha)$ sürer. x_i tabloya eklenen i. eleman, $X_{ij} = 1$ (x_i ve x_j aynı slota düşerse), değilse 0. $E[X_{ij}] = 1/m$.
  - Ortalama inceleme sayısı: $\frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}\frac{1}{m}\right) = 1+\frac{1}{nm}\sum_{i=1}^{n}(n-i) = 1+\frac{n-1}{2m} = 1+\frac{\alpha}{2}-\frac{\alpha}{2n} = \Theta(1+\alpha)$.
- İyi hash fonksiyonu ve stringler (ekrandan, 01:50:00)
  - Anahtarların dağılımı rastgeleye yakın olmalıdır; "pt" ve "pts" gibi birbirine çok yakın stringler sorun çıkarmamalıdır. Anahtarlar $[0,1)$ aralığındaki sayılarsa m ile çarpmak düzgün dağılım verir.
  - Stringler radix gösterimiyle doğal sayıya çevrilir. Radix 128 ile "pt": p = 112, t = 116, $112 \cdot 128 + 116 \cdot 128^0 = 14452$.
- Bölme yöntemi (division method) (ekrandan, 01:52:55)
  - $h(k) = k \bmod m$. m seçimi kritiktir: 2'nin kuvvetlerine yakın olmayan bir asal seçilir.
  - Örnek: $n = 2000$ eleman, ortalama 3 elemanlık zincir göze alınıyor; $2000/3 \approx 667$'ye yakın ve 2'nin kuvvetlerinden uzak asal $m = 701$.
- Çarpma yöntemi (multiplication method) (ekrandan, 02:00:30)
  - Adımlar: k bir $0 < A < 1$ sayısıyla çarpılır, kA'nın ondalık kısmı alınır, m ile çarpılır, aşağı yuvarlanır: $h(k) = \lfloor m(kA \bmod 1)\rfloor$. Burada m'nin seçimi kritik değildir, A'nın seçimi önemlidir.
  - Önerilen $A = (\sqrt{5}-1)/2 \approx 0{,}618033$ (altın oran).
  - Örnek: $m = 10000$, $k = 123456$: $123456 \cdot 0{,}618033 \approx 76300{,}0041151$; ondalık kısım $0{,}0041151$; $10000 \cdot 0{,}0041151 = 41{,}151$; $h(k) = 41$.

- Açık adresleme (open addressing) (ekrandan, 02:07:20)
  - Bütün elemanlar tablonun kendisinde tutulur. Slot doluysa başka slot denenir; deneme sırası iki parametreli hash fonksiyonuyla belirlenir: $h(k, i)$, $i = 0, 1, \ldots, m-1$ deneme numarasıdır.
  - HASH-INSERT: $i = 0$; `repeat` $j = h(k, i)$; $T[j]$ boşsa $T[j] = k$, j döner; değilse $i = i + 1$; `until` $i = m$; sonra "hash table overflow" hatası.
  - HASH-SEARCH: $i = 0$; `repeat` $j = h(k, i)$; $T[j] = k$ ise j döner; $i = i + 1$; `until` $T[j] = NIL$ ya da $i = m$.
  - Silme sorunludur: silinen yere NIL konursa arama erken biter. Onun yerine DELETED işareti konur; insert bu slotu boş sayar. Çok silme varsa arama süresi yük faktöründen çok silinen elemanlara bağlı olur; çok silme gerekiyorsa zincirleme daha uygundur.
- Açık adresleme teknikleri: hepsi tek parametreli yardımcı hash fonksiyonu $h'(k)$ kullanır.
  - Doğrusal deneme (linear probing) (ekrandan, 02:17:55): $h(k, i) = (h'(k) + i) \bmod m$, $h'(k) = k \bmod m$. $m = 7$: 9 → slot 2; 16 → 2 dolu, $(2 + 1) \bmod 7 = 3$; 23 → 4. Gerçeklemesi kolaydır ama kümelenme (clustering) oluşur, aramalar uzar.
  - Karesel deneme (quadratic probing) (ekrandan, 02:24:25): $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$; başka bir biçim $(h'(k) + i^2) \bmod m$. $c_1 = 2$, $c_2 = 1$, $m = 7$: 9 → 2; 16 → $2 + 2 + 1 = 5$; 23 → 2 ve 5 dolu, $i = 2$ için $2 + 4 + 4 = 10$. Kümelenme azalır ama tamamen kalkmaz; aynı başlangıç konumlu anahtarlar aynı diziyi izler. Bütün tablonun kullanılması için c_1, c_2, m dikkatle seçilmelidir.
  - Çift hashing (double hashing) (ekrandan, 02:35:40): iki hash fonksiyonu birlikte kullanılır; ürettiği deneme permütasyonları rastgeleye yakın olduğu için en iyi sonuç verir. $h_1(k) = k \bmod m$ (m tablo boyutu), $h_2(k) = 1 + (k \bmod m')$, m' biraz küçük ve asal. $m = 13$, $m' = 11$, $k = 14$: $h_1 = 1$, $h_2 = 4$; slot 1 dolu (79), slot 5 dolu (98), $1 + 2 \cdot 4 = 9$ boş, 14 slot 9'a yerleşir.

Hocanın Özellikle Vurguladığı Kısımlar

- Tahmin tümevarımın en önemli ve en zor adımıdır
  - Tahmin yanlışsa ispatta bu açıkça görünür: $T(n) \leq cn$ tahmininde $cn + n$ çıkar ve eşitsizlik hiçbir pozitif n için sağlanmaz.
- Eksi işaretli fazlalık ispatı kolaylaştırır
  - $cn \log n - n(c-1)$ gibi, varsayılan sınırdan pozitif bir şey çıkarılıyorsa ispat tamamdır; artı işaretli fazlalık kalırsa daha dikkatli düşünmek ve koşul bulmak gerekir. Medyan örneğinde ifade bu yüzden $cn - (\ldots)$ biçimine getirildi.
- Medyanda sabit eşiğin anlamı
  - $c \geq 20a$, $n \geq 120$ koşulu, algoritmada $O(1)$'de çözülecek küçük girdi eşiğinin nereye konacağını da gösterir.
- Zincirlemede çift yönlü bağlı liste
  - Silmeyi $O(1)$ yapan, düğümün öncekini ve sonrakini birlikte tutmasıdır. Tek yönlü listede aynı iş $O(n)$'e çıkar.
- Hash fonksiyonunun kalitesi
  - Rastgeleye ne kadar yakın dağıtırsa o kadar az çakışma olur. Bölme yönteminde m, çarpma yönteminde A kritiktir.
- Açık adreslemede silme
  - Silme sık yapılacaksa açık adresleme iyi bir çözüm değildir; zincirleme tercih edilir.
- Kümelenme
  - Doğrusal deneme kolaydır ama kümelenme aramayı uzatır; karesel deneme bunu azaltır, çift hashing en iyi dağılımı verir.

Kısa Tekrar Notları

- Master teoremi eşit boyutlu alt problemler ister; medyan bulmada tümevarımla yerine koyma kullanılır.
- Yöntem: tahmin et, $m < n$ için doğru kabul et, n için göster, c ve n_0 bul.
- Merge sort: $T(n) \leq cn \log n - n(c-1)$; $c = 2$, $n_0 = 2$.
- $T(n) \leq cn$ tahmini merge sort için yanlış: $cn + n \not\leq cn$.
- Medyan bulma: $T(n) \leq cn - (cn/10 - 6c - an)$; $n \geq 120$, $c \geq 20a$ ile $O(n)$.

- Doğrudan adresleme: $O(1)$ işlemler, büyük evrende bellek sorunu.
- Yük faktörü $\alpha = n/m$; basit düzgün hashing varsayımıyla başarısız ve başarılı arama $\Theta(1+\alpha)$.
- Zincirleme: başa ekleme $O(1)$, çift yönlü listede silme $O(1)$, en kötü arama $O(n)$.
- Bölme: $h(k) = k \bmod m$, m asal ve 2'nin kuvvetlerinden uzak (örnek $m = 701$).
- Çarpma: $h(k) = \lfloor m(kA \bmod 1) \rfloor$, $A \approx 0{,}618$; $h(123456) = 41$ ($m = 10000$).
- Açık adresleme: doğrusal $(h'(k) + i) \bmod m$, karesel $(h'(k) + c_1 i + c_2 i^2) \bmod m$, çift hashing $(h_1(k) + i\, h_2(k)) \bmod m$.

Detaylı Açıklamalar

- **Tümevarımda öğrenci sorusu.** Bir öğrenci, tümevarımda "$n+1$ için" göstermek yerine neden $n/2$ üzerinden gidildiğini sordu. Hoca, bağıntı $n-1$, $n-2$ diye değil ikiye bölerek ilerlediği için varsayımın $m = n/2$ için kullanıldığını; sonunda elde edilen eşitsizliğin n'nin hangi değerleri için tuttuğunu zaten gösterdiğini; başlangıç değerlerinde de c ve n_0 denenerek kontrol edildiğini söyledi.
- **Merge sort ispatının ayrıntısı** (ekrandan, 00:26:40). $\log(n/2) = \log n - \log 2$ ve $\log_2 2 = 1$ kullanılır: $cn \log \frac{n}{2} + n = cn \log n - cn + n = cn \log n - n(c-1)$. Başlangıçta $n = 2$ için $T(2) = 2T(1) + 2 = 4$, $n = 3$ için $T(3) = 2T(1) + 3 = 5$. $c = 2$: $T(2) = 4 \le 2 \cdot 2 \log 2 = 4$.
- **Ders sonrası not (merge sort ispatı).** Hoca eksi işaretli terim çıktığı için ispatın bittiğini, n ve c pozitif olduğu için yeterli olduğunu söyledi. $cn \log n - n(c-1) \le cn \log n$ için $c - 1 \ge 0$, yani $c \ge 1$ gerekir; seçilen $c = 2$ bunu sağlar. $c = 2$ ile $T(n) \le 2n \log_2 n$ eşitsizliği $T(1) = 1$ ve taban fonksiyonlu bağıntı için $2 \le n < 5000$ aralığında python ile doğrulandı.
- **Medyan ispatının ayrıntısı** (ekrandan, 00:40:00 ve 00:42:40). $9cn/10$, $cn - cn/10$ diye yazılıp $cn + (-cn/10 + 6c + an)$ ve sonra $cn - (cn/10 - 6c - an)$ biçimine getirilir; merge sort'taki gibi "sınır eksi pozitif bir şey" yapısı kurulur. $cn/10 \ge 6c + an \Rightarrow cn \ge 60c + 10an \Rightarrow c \ge 60c/n + 10a$. Eşitlik durumu düşünülerek $n = 120$, $c = 20a$ denenir: $60 \cdot 20a/120 + 10a = 20a$. $c = 20a$ için $cn/10 - 6c - an \ge 0$ koşulu $n = 120, 140, 500, 10^6$ için python ile kontrol edildi; n büyüdükçe gereken c alt sınırı $(10an/(n-60))$ küçülür.
- **Bağlı liste kodları** (ekrandan, 01:00:00). Sona eklemede head'in kaybolmaması için `current` gibi yedek bir işaretçiyle sona kadar gidilir, yeni düğüme yer açılır, değeri yazılır, next'i NULL yapılır. Başa eklemede yeni düğümün next'i eski head'i gösterir ve yeni head döndürülür. Değere göre silme kodu: `while ((current->val != val) && (current->next != NULL))` ile `before = current; current = current->next;` ilerlenir; değer bulunamazsa "silinmek istenen eleman listede yok" yazılır; bulunursa `before->next = current->next; free(current);`.
- **Neden çift yönlü liste?** Hoca öğrencilere sordu ve kendisi cevapladı: silmeyi hızlandırdığı için. Silinecek düğüm verildiğinde önceki ve sonraki düğümün adresi elde vardır; iki bağlantı güncellenir, $O(1)$. Tek yönlü listede düğüm verilse bile önceki düğümü bulmak için zincir baştan taranır; bütün elemanlar aynı slottaysa $O(n)$.
- **Başarısız arama sezgisi.** Zincire sırayla x_1, x_2, x_3 eklendiğinde her biri listenin başına girer, liste $x_3 \to x_2 \to x_1$ olur. x_1'i bulmak için ondan sonra eklenenler ve kendisi, yani zincirdeki eleman sayısı kadar inceleme gerekir. Başarısız arama zincirin sonuna kadar gider; zincirde ortalama α eleman, artı hash hesaplaması: $\Theta(1+\alpha)$.
- **Başarılı arama ispatı** (ekrandan, 01:33:40 ve 01:40:00). $1 + \sum_{j=i+1}^{n} X_{ij}$, x_i aranırken yapılan inceleme sayısıdır: 1, x_i'nin kendisi; toplam, x_i'den sonra eklenip aynı slota düşen (listede önüne geçen) elemanlar. $1/n$ ile çarpmak ortalamayı verir. Beklenen değerde $E[X_{ij}] = 1/m$ konur; $\sum_{j=i+1}^{n} 1/m = (n-i)/m$; $\sum_{i=1}^{n}(n-i) = n^2 - n(n+1)/2$. Sonuç $1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$; sabit katsayılar atılınca $\Theta(1+\alpha)$. Formül $n = 50$, $m = 7$ için python ile doğrulandı.

- **Ders sonrası not (bölme yöntemi örneği).** Hoca "3 kez başarısız arama yapılmasını sorun etmemek" dedi (01:52:23). Kastedilen, başarısız bir aramada ortalama 3 eleman incelemeyi göze almaktır, yani yük faktörünün yaklaşık 3 olması: $2000/3 \approx 666{,}7$, bu yüzden $m = 701$ (ekrandan, 01:52:55).
- **Çarpma yöntemi örneği** python ile doğrulandı: $123456 \cdot 0{,}6180339887 = 76300{,}00411\ldots$, $\lfloor 10000 \cdot 0{,}0041151 \rfloor = 41$. Hoca altın oran önerisini algoritma analizinin babası sayılan, Turing ödüllü bir araştırmacıya bağladı; bu büyük olasılıkla Donald Knuth'tur (altyazıda ad bozuk).
- **Ders sonrası not (karesel deneme örneği).** Hoca 23 için $i = 2$'de $2 + 2 \cdot 2 + 2^2 = 10$ buldu ve elemanı tabloda "bir yere" yerleştirdi (ekrandan, 02:24:25). Fonksiyonda mod m vardır: $10 \bmod 7 = 3$, 23 slot 3'e yerleşir (slot 3 boştur). Bu parametrelerle ($c_1 = 2$, $c_2 = 1$, $m = 7$) $h'(k) = 2$ olan bir anahtarın deneme dizisi $i = 0, \ldots, 6$ için 2, 5, 3, 3, 5, 2, 1 olur (python ile); yalnız dört slot denenir. Hocanın "c_1, c_2, m tablonun tamamı kullanılacak şekilde seçilmeli" uyarısı bununla ilgilidir.
- **Çift hashing örneği** (ekrandan, 02:30:00 ve 02:35:40). Tabloda önceden dolu slotlar: 1 (79), 4 (69), 5 (98), 7 (72), 11 (50). $k = 14$: $h_1 = 14 \bmod 13 = 1$, $h_2 = 1 + (14 \bmod 11) = 4$. Denemeler 1 (dolu), $1 + 4 = 5$ (dolu), $1 + 2 \cdot 4 = 9$ (boş). Hoca $m' = m - 1 = 12$'nin çift olacağını, asal olması daha iyi olduğu için 11 seçtiğini söyledi.
- **Açık adreslemede silme sorusu.** Hoca silmenin nasıl yapılacağını sordu ve cevapladı: NIL koymak aramayı erken bitirir, çünkü arama NIL görünce durur ama aynı deneme dizisinde ileride yerleştirilmiş elemanlar olabilir. DELETED işareti bu sorunu çözer, insert onu boş kabul eder.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.